

This allows the production code to be refactored and evolved in ways that don't affect the tests. It also allows the tests to be refactored and evolved in ways that don't affect the production code.

This separation of evolution is necessary because as time passes, the tests tend to become increasingly more concrete and specific. In contrast, the production code tends to become increasingly more abstract and general. Strong structural coupling prevents—or at least impedes—this necessary evolution, and prevents the production code from being as general, and flexible, as it could be.

SECURITY

The superpowers of the testing API could be dangerous if they were deployed in production systems. If this is a concern, then the testing API, and the dangerous parts of its implementation, should be kept in a separate, independently deployable component.

CONCLUSION

Tests are not outside the system; rather, they are parts of the system that must be well designed if they are to provide the desired benefits of stability and regression. Tests that are not designed as part of the system tend to be fragile and difficult to maintain. Such tests often wind up on the maintenance room floor—discarded because they are too difficult to maintain.

CLEAN EMBEDDED ARCHITECTURE



By James Grenning

A while ago I read an article entitled “The Growing Importance of Sustaining Software for the DoD”¹ on Doug Schmidt’s blog. Doug made the following claim:

“Although software does not wear out, firmware and hardware become obsolete, thereby requiring software modifications.”

It was a clarifying moment for me. Doug mentioned two terms that I would have thought to be obvious—but maybe not. *Software* is this thing that can have a long useful life, but *firmware* will become obsolete as hardware evolves. If you have spent any time in embedded systems development, you know the hardware is continually evolving and being improved. At the same time, features are added to the

new “software,” and it continually grows in complexity.

I’d like to add to Doug’s statement:

Although software does not wear out, it can be destroyed from within by unmanaged dependencies on firmware and hardware.

It is not uncommon for embedded software to be denied a potentially long life due to being infected with dependencies on hardware.

I like Doug’s definition of firmware, but let’s see which other definitions are out there. I found these alternatives:

- “Firmware is held in non-volatile memory devices such as ROM, EPROM, or flash memory.” (<https://en.wikipedia.org/wiki/Firmware>)
- “Firmware is a software program or set of instructions programmed on a hardware device.” (<https://techterms.com/definition/firmware>)
- “Firmware is software that is embedded in a piece of hardware.” (<https://www.lifewire.com/what-is-firmware-2625881>)
- Firmware is “Software (programs or data) that has been written onto read-only memory (ROM).” (<http://www.webopedia.com/TERM/F/firmware.html>)

Doug’s statement makes me realize that these accepted definitions of firmware are wrong, or at least obsolete. Firmware does not mean code lives in ROM. It’s not firmware because of where it is stored; rather, it is firmware because of what it depends on and how hard it is to change as hardware evolves. Hardware does evolve (pause and look at your for phone for evidence), so we should structure our embedded code with that reality in mind.

I have nothing against firmware, or firmware engineers (I’ve been known to write some firmware myself). But what we really need is less firmware and more software. Actually, I am disappointed that firmware engineers write so much firmware!

Non-embedded engineers also write firmware! You non-embedded developers essentially write firmware whenever you bury SQL in your code or when you spread platform dependencies throughout your code. Android app developers write firmware when they don’t separate their business logic from the Android API.

I’ve been involved in a lot of efforts where the line between the product code (the software) and the code that interacts with the product’s hardware (the firmware) is

fuzzy to the point of nonexistence. For example, in the late 1990s I had the fun of helping redesign a communications subsystem that was transitioning from time-division multiplexing (TDM) to voice over IP (VOIP). VOIP is how things are done now, but TDM was considered the state of the art from the 1950s and 1960s, and was widely deployed in the 1980s and 1990s.

Whenever we had a question for the systems engineer about how a call should react to a given situation, he would disappear and a little later emerge with a very detailed answer. “Where did he get that answer?” we asked. “From the current product’s code,” he’d answer. The tangled legacy code was the spec for the new product! The existing implementation had no separation between TDM and the business logic of making calls. The whole product was hardware/technology dependent from top to bottom and could not be untangled. The whole product had essentially become firmware.

Consider another example: Command messages arrive to this system via serial port. Unsurprisingly, there is a message processor/dispatcher. The message processor knows the format of messages, is able to parse them, and can then dispatch the message to the code that can handle the request. None of this is surprising, except that the message processor/dispatcher resides in the same file as code that interacts with a UART² hardware. The message processor is polluted with UART details. The message processor could have been software with a potentially long useful life, but instead it is firmware. The message processor is denied the opportunity to become software—and that is just not right!

I’ve known and understood the need for separating software from hardware for a long time, but Doug’s words clarified how to use the terms *software* and *firmware* in relationship to each other.

For engineers and programmers, the message is clear: Stop writing so much firmware and give your code a chance at a long useful life. Of course, demanding it won’t make it so. Let’s look at how we can keep embedded software architecture clean to give the software a fighting chance of having a long and useful life.

APP-TITUDE TEST

Why does so much potential embedded software become firmware? It seems that most of the emphasis is on getting the embedded code to work, and not so much emphasis is placed on structuring it for a long useful life. Kent Beck describes three

activities in building software (the quoted text is Kent's words and the italics are my commentary):

1. "First make it work." *You are out of business if it doesn't work.*
2. "Then make it right." *Refactor the code so that you and others can understand it and evolve it as needs change or are better understood.*
3. "Then make it fast." *Refactor the code for "needed" performance.*

Much of the embedded systems software that I see in the wild seems to have been written with "Make it work" in mind—and perhaps also with an obsession for the "Make it fast" goal, achieved by adding micro-optimizations at every opportunity. In *The Mythical Man-Month*, Fred Brooks suggests we "plan to throw one away." Kent and Fred are giving virtually the same advice: Learn what works, then make a better solution.

Embedded software is not special when it comes to these problems. Most non-embedded apps are built just to work, with little regard to making the code right for a long useful life.

Getting an app to work is what I call the *App-titude test* for a programmer. Programmers, embedded or not, who just concern themselves with getting their app to work are doing their products and employers a disservice. There is much more to programming than just getting an app to work.

As an example of code produced while passing the App-titude test, check out these functions located in one file of a small embedded system:

[Click here to view code image](#)

```
ISR(TIMER1_vect) { ... }
ISR(INT2_vect) { ... }
void btn_Handler(void) { ... }
float calc_RPM(void) { ... }
static char Read_RawData(void) { ... }
void Do_Average(void) { ... }
void Get_Next_Measurement(void) { ... }
void Zero_Sensor_1(void) { ... }
void Zero_Sensor_2(void) { ... }
void Dev_Control(char Activation) { ... }
char Load_FLASH_Setup(void) { ... }
void Save_FLASH_Setup(void) { ... }
void Store_DataSet(void) { ... }
```

```
float bytes2float(char bytes[4]) { ... }
void Recall_DataSet(void) { ... }
void Sensor_init(void) { ... }
void uC_Sleep(void) { ... }
```

That list of functions is in the order I found them in the source file. Now I'll separate them and group them by concern:

- Functions that have domain logic

[Click here to view code image](#)

```
float calc_RPM(void) { ... }
void Do_Average(void) { ... }
void Get_Next_Measurement(void) { ... }
void Zero_Sensor_1(void) { ... }
void Zero_Sensor_2(void) { ... }
```

- Functions that set up the hardware platform

[Click here to view code image](#)

```
ISR(TIMER1_vect) { ... }*
ISR(INT2_vect) { ... }
void uC_Sleep(void) { ... }
Functions that react to the on off button press
void btn_Handler(void) { ... }
void Dev_Control(char Activation) { ... }
A Function that can get A/D input readings from the hardware
static char Read_RawData(void) { ... }
```

- Functions that store values to the persistent storage

[Click here to view code image](#)

```
char Load_FLASH_Setup(void) { ... }
void Save_FLASH_Setup(void) { ... }
void Store_DataSet(void) { ... }
float bytes2float(char bytes[4]) { ... }
void Recall_DataSet(void) { ... }
```

- Function that does not do what its name implies

[Click here to view code image](#)

```
void Sensor_init(void) { ... }
```

Looking at some of the other files in this application, I found many impediments to understanding the code. I also found a file structure that implied that the only way to